

VULNERABILITY ASSESSMENT REPORT

For: Future Interns - Cyber Security Task 1

Target: Damn Vulnerable Web Application
(DVWA)

URL: <http://localhost>

Prepared by: NORAH CYPRIAN NGILIULE

CIN ID: FIT/MAY26/CS8136

Date: 9th MAY,2026

1.Introduction

This report documents the Vulnerability Assessment conducted as part of Task 1 under the Cyber Security Internship Program by Future Interns. The objective of this task is to identify common web security vulnerabilities in a live website, classify risk levels, and provide clear remediation steps.

2.Scope And Methodology

Assessment Type: Passive Vulnerability Assessment

Target URL: <http://localhost>

Application: Damn Vulnerable Web Application (DVWA)

Environment: Oracle VirtualBox Manager

Allowed activities:

1. Target identification
2. Passive scanning and reconnaissance
3. Manual inspection using browser tools
4. Risk classification
5. Documentation and reporting

Not performed:

Exploitation

SQL Injection testing

Brute force attacks

Denial-of-Service attacks

All testing was conducted in read-only mode, respecting ethical guidelines.

3.Tools Used

- Nmap - Port scanning and service detection
- OWASP ZAP - Passive Vulnerability Scanning
- Firefox Developer Tools - Header and Cookies analysis
- WhatWeb - Web technology fingerprinting
- Canva for report design.
- GitHub

3.EXECUTIVE SUMMARY

During this read only vulnerability assessment, five security issues were identified.

Risk Level Distribution

- High Risk: 0 findings
- Medium Risk: 3 findings
- Low Risk: 2 findings

Key Findings Overview

During this read only vulnerability assessment, five security issues were identified and classified according to their potential impact and risk level:

1. **Missing Content Security Policy (CSP) Header — *Medium Risk***

The application does not implement a Content Security Policy header, increasing the risk of Cross-Site Scripting (XSS) and other content injection attacks.

2. **MySQL Database Exposed on Port 3306 — *Medium Risk***

The MySQL service is publicly accessible through port 3306, which may allow unauthorized access attempts against the database server.

3. **Missing X-Frame-Options Header — *Medium Risk***

The absence of the X-Frame-Options header may expose the application to clickjacking attacks by allowing pages to be embedded within external frames.

4. **Cookies Missing Secure and SameSite Attributes — *Low Risk***

Session cookies are configured without Secure and SameSite attributes, which may increase the risk of session hijacking and cross-site request forgery (CSRF) attacks.

5. **Server Version Information Disclosure — *Low Risk***

The web server reveals version information in HTTP headers, potentially providing attackers with useful information for targeted attacks.

Priority Recommendations

1. Implement essential security headers, including: Content-Security-Policy (CSP), X-Frame-Options,X-Content-Type-Options.
2. Restrict MySQL access by configuring the database service to listen only on localhost or trusted internal networks.
- 3.Configure session cookies with the Secure and SameSite attributes to improve session protection.
- 4.Disable server version disclosure within HTTP response headers to reduce information leakage.

FINDINGS SUMMARY

	Findings	Risk level
1	Missing Content-Security-Policy (CSP) Header	Medium Risk
2	MySQL Database Exposed on Port 3306	Medium Risk
3	Missing X-Frame-Options Header	Medium Risk
4	Cookie Missing Secure & SameSite Attributes	Low Risk
5	Server Version Information Disclosure	Low Risk

4.DETAILS FINDINGS

FINDING 1: Missing Content-Security-Policy Header

Risk Level: Medium

Description

The application does not implement a Content-Security-Policy (CSP) header. CSP is an important security mechanism that helps protect web applications against Cross-Site Scripting (XSS) and malicious content injection attacks.

Evidence

HTTP response header inspection using browser developer tools showed that the Content-Security-Policy header was not present in the server response.

Impact

Without CSP protection, attackers may be able to inject and execute malicious scripts within the application which could result in Session hijacking, Credential theft, Unauthorized user actions, Browser-based attacks.

Remediation

Configure the web server to include a secure CSP header.

Example: Content-Security-Policy: default-src 'self';

Evidence Files: headers.txt

FINDING 2: MySQL Database Exposed on Port 3306

Risk Level: Medium

Description

The MySQL database service was found accessible through port 3306. Database services should not be publicly exposed unless specifically required.

Evidence

Network scanning identified: 3306/tcp open mysql
indicating that the MySQL service is reachable externally.

Impact

An exposed MySQL service increases the risk of: Unauthorized access attempts, Brute-force password attacks, Database enumeration, Potential data compromise if weak credentials exist.

Remediation

Restrict database access by the following steps:

Steps

1. Edit the MySQL configuration file: `sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf`
2. Find or add the bind-address directive: `bind-address = 127.0.0.1`
3. Restart the MySQL service: `sudo systemctl restart mysql`
 - After this change, only processes running on the same machine can access the database.
4. Blocking public access using firewall rules
5. Enforcing strong authentication policies

Evidence Files: nmap.txt

FINDING 3: Missing X-Frame-Options Header

Risk Level: Medium

Description

The application does not include the X-Frame-Options security header. This header helps protect against clickjacking attacks by preventing pages from being loaded inside frames or iframes on malicious websites.

Evidence

Browser developer tools inspection confirmed that the X-Frame-Options header was missing from the HTTP response headers.

Impact

Attackers may trick users into interacting with invisible or disguised elements embedded within malicious websites potentially causing: -

- Unauthorized actions
- Credential theft
- Fraudulent transactions
- User interface manipulation

Remediation

Configure the web server to include one of the following headers:

X-Frame-Options: SAMEORIGIN

or

X-Frame-Options: DENY

Evidence Files: Headers.txt

FINDING 4: Cookie Missing Secure and SameSite Attributes

Risk Level: Low

Description

Session cookies were identified without the Secure and SameSite attributes enabled. These attributes help improve cookie security and reduce the risk of session-based attacks.

Evidence

Cookie analysis from browser developer tools showed:

Set-Cookie: PHPSESSID="75988d634282bd3af88cf44a31b8ccfc"; path="/";
HttpOnly

- The cookie did not contain: Secure and SameSite

Impact

Missing cookie protections may increase the risk of:

- Session hijacking
- Cross-Site Request Forgery (CSRF)
- Cookie interception over insecure connections

Remediation

Configure session cookies with secure attributes:

Set-Cookie: PHPSESSID="75988d634282bd3af88cf44a31b8ccfc"; Secure;
HttpOnly; SameSite=Strict

Recommended attributes:

- Secure → Sends cookies only over HTTPS
- SameSite → Prevents unauthorized cross-site requests
- HttpOnly → Blocks JavaScript access to cookies

Evidence Files: Cookies.txt

FINDING 5: Server Version Information Disclosure

Risk Level: Low

Description

The web server exposes software version information within HTTP response headers. This information may help attackers identify known vulnerabilities associated with the server version.

Evidence

HTTP response headers revealed:

Server: Apache/2.4.65 (Debian)

Impact

Information disclosure may assist attackers in:

- Identifying outdated software versions
- Researching known vulnerabilities
- Launching targeted attacks

Remediation

Disable or minimize server version disclosure within server configuration files.

For apache

ServerTokens Prod

ServerSignature Off

For nginx

server_tokens off;

- keep the server software updated with the latest stable security patches.

Evidence Files: whatweb.txt, headers.txt

5. CONCLUSION AND RECOMMENDATIONS

The vulnerability assessment identified several security weaknesses within the target web application environment. Although no active exploitation was performed during testing the identified issues could potentially increase the risk of unauthorized access, information disclosure, session attacks, and client-side exploitation if left unaddressed.

Most of the findings are related to missing security configurations and exposed services which are common but important security concerns in web application environments. The overall security posture of the application can be significantly improved through proper server hardening, secure cookie configuration, and implementation of recommended HTTP security headers.

Recommendations Summary

To improve the security of the application and reduce potential attack surfaces the following actions are recommended:

1. Implement essential HTTP security headers including:
 - Content-Security-Policy (CSP)
 - X-Frame-Options
 - X-Content-Type-Options
2. Restrict MySQL database access to localhost or trusted internal networks only.
3. Configure session cookies with:
 - Secure
 - HttpOnly
 - SameSite attributes
4. Disable unnecessary server version disclosure in HTTP response headers.

5.Regularly update and patch web server software, database systems, and application dependencies.

6.Conduct periodic vulnerability assessments and security reviews to identify and remediate newly discovered risks.

- By implementing these recommendations, the organization can strengthen its overall security posture and reduce exposure to common web-based attacks.
- The target (DVWA) is a deliberately vulnerable web application used exclusively for authorized security training and assessment purposes.

8. PROOF OF CONCEPT

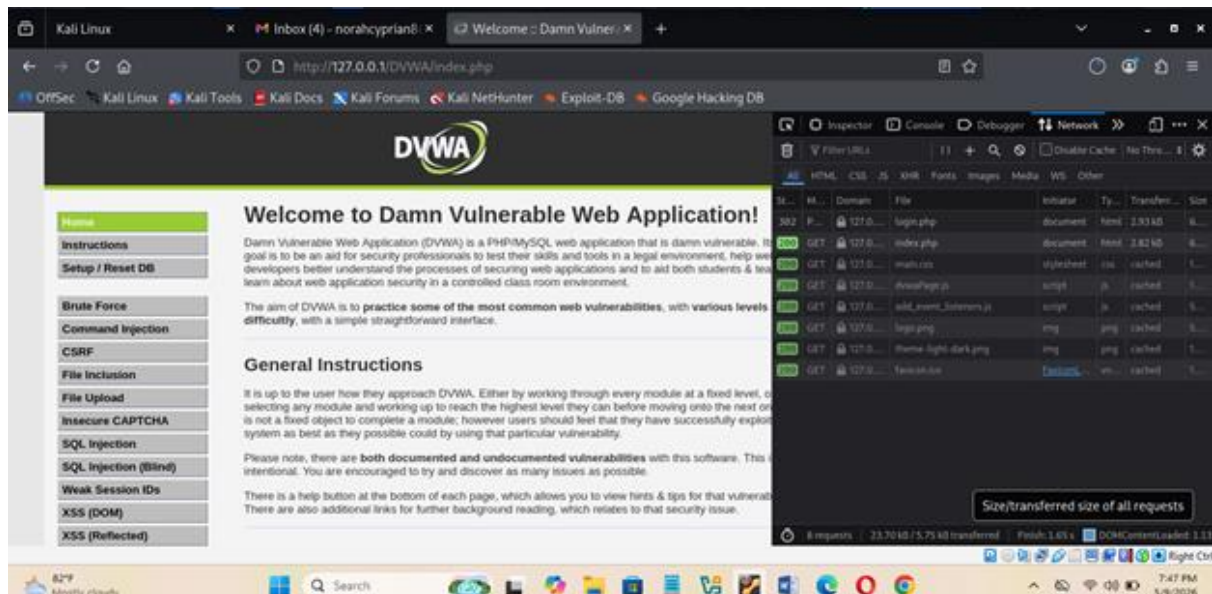
. nmap -sC - Pn 127.0.0.1

```
root@kali:~# nmap -sC -Pn 127.0.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-09 11:09 EDT
Nmap scan report for localhost (127.0.0.1)
Host is up (0.000011s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE
80/tcp    open  http
|_http-title: Apache2 Debian Default Page: It works
3306/tcp  open  mysql
|_ssl-cert: Subject: commonName=MariaDB Server
|_Not valid before: 2026-05-09T14:54:19
|_Not valid after: 2026-05-06T14:54:19
|_ssl-date: TLS randomness does not represent time
|_mysql-info:
|_Protocol: 10
|_Version: 11.0.3-MariaDB-1+b1 from Debian
|_Thread ID: 43
|_Capabilities flags: 65534
|_Some Capabilities: SupportsAuth, FoundRows, Speaks41ProtocolOld, IgnoreSigpipes, LongColumnFlag, SupportsCompression, SupportsTransactions, SwitchToSSLAfterHandshake, InteractiveClient, SupportsLoadDataLocal, ConnectWithDatabase, DontAllowDatabaseTableColumn, ODBCClient, Speaks41ProtocolNew, IgnoreSpaceBeforeParenthesis, SupportsAuthPlugins, SupportsMultipleStatements, SupportsMultipleResults
|_Status: Autocommit
|_Salt: h1gfFjnnayKv.n.sD:
|_Auth Plugin Name: mysql_native_password
Nmap done: 1 IP address (1 host up) scanned in 2.02 seconds
```

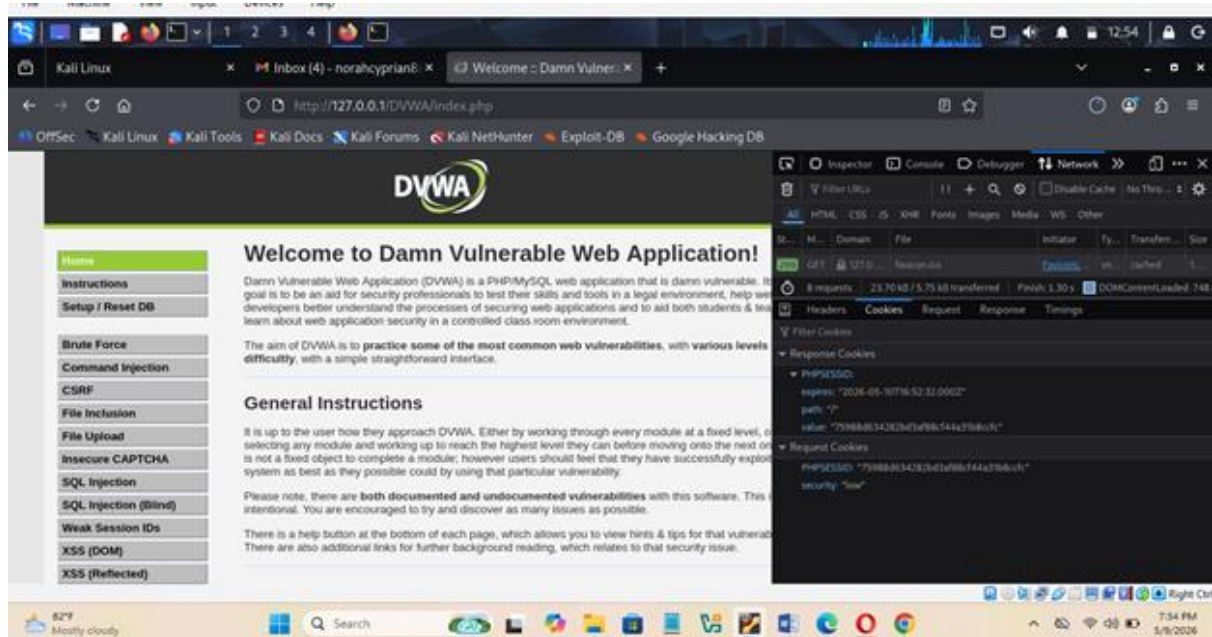
. Whatweb

```
root@kali:~# whatweb http://localhost
http://localhost [200 OK] Apache[2.4.65], HTTPServer[Debian Linux][Apache/2.4.65 (Debian)], IP[::1], Title[Apache2 Debian Default Page: It works]
```

. Firefox Developer Tools



Cookies



Headers

The screenshot shows a Kali Linux desktop environment with a web browser open to the DVWA homepage. The browser's developer tools are open, showing the Network tab. A POST request to `http://127.0.0.1/DVWA/login.php` is selected. The response headers are displayed, showing a 302 Found status and various cookies.

Response Headers (355 B)

- Cache-Control: no-store, no-cache, must-revalidate
- Connection: Keep-Alive
- Content-Length: 0
- Content-Type: text/html; charset=UTF-8
- Date: Sat, 09 May 2020 16:52:32 GMT
- Expires: Thu, 19 Nov 1981 08:52:00 GMT
- Keep-Alive: timeout=5, max=100
- Location: index.php
- Pragma: no-cache
- Server: Apache/2.4.18 (Ubuntu)
- Set-Cookie: PHPSESSID=75988634282b3d3f8b44a378bdcf; expires=Sun, 10 May 2020 16:52:32 GMT; Max-Age=600; path=/

The screenshot shows a Kali Linux desktop environment with a web browser open to the DVWA homepage. The browser's developer tools are open, showing the Network tab. A GET request to `http://127.0.0.1/DVWA/index.php` is selected. The response headers are displayed, showing a 200 OK status and various cookies.

Response Headers (355 B)

- Cache-Control: no-store, no-cache, must-revalidate
- Connection: Keep-Alive
- Content-Length: 0
- Content-Type: text/html; charset=UTF-8
- Date: Sat, 09 May 2020 16:52:32 GMT
- Expires: Thu, 19 Nov 1981 08:52:00 GMT
- Keep-Alive: timeout=5, max=100
- Location: index.php
- Pragma: no-cache
- Server: Apache/2.4.18 (Ubuntu)
- Set-Cookie: PHPSESSID=75988634282b3d3f8b44a378bdcf; expires=Sun, 10 May 2020 16:52:32 GMT; Max-Age=600; path=/

Screenshot of a Kali Linux desktop environment showing a web browser displaying the Damn Vulnerable Web Application (DVWA) homepage. The browser's address bar shows the URL `http://127.0.0.1/DVWA/index.php`. The page title is "Welcome to Damn Vulnerable Web Application!". The page content includes a sidebar with navigation links (Home, Instructions, Setup / Reset DB, Bruteforce, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, SQL Injection (Blind), Weak Session IDs, XSS (DOM), XSS (Reflected)) and a main content area with a welcome message and general instructions. The network tab in the browser's developer tools is open, showing a list of requests and the selected request's details, including headers and cookies.

Welcome to Damn Vulnerable Web Application!

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL web application that is damn vulnerable. Its goal is to be an aid for security professionals to test their skills and tools in a legal environment, help web developers better understand the processes of securing web applications and to aid both students & teachers learn about web application security in a controlled class room environment.

The aim of DVWA is to practice some of the most common web vulnerabilities, with various levels of difficulty, with a simple straightforward interface.

General Instructions

It is up to the user how they approach DVWA. Either by working through every module at a fixed level, or selecting any module and working up to reach the highest level they can before moving onto the next or is not a fixed object to complete a module; however users should feel that they have successfully exploited system as best as they possible could by using that particular vulnerability.

Please note, there are **both documented and undocumented vulnerabilities** with this software. This is intentional. You are encouraged to try and discover as many issues as possible.

There is a help button at the bottom of each page, which allows you to view hints & tips for that vulnerability. There are also additional links for further background reading, which relates to that security issue.

Network Tab Details:

- Filter: All
- Request: GET /index.php
- Response: 200 OK
- Headers:
 - Connection: keep-alive
 - Content-Length: 88
 - Content-Type: application/javascript
 - Cookie: security=low; PHPSESSID=79888834302b3d3e8b44a218d8c
 - Host: 127.0.0.1
 - Origin: http://127.0.0.1
 - Priority: u=0,1
 - Referer: http://127.0.0.1/DVWA/index.php
 - Sec-Fetch-Dest: document
 - Sec-Fetch-Mode: navigate
 - Sec-Fetch-Site: same-origin
 - Sec-Fetch-User: ?1
 - Upgrade-Insecure-Requests: 1
 - User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:78.0) Gecko/20100101 Firefox/78.0